简历技术扩展 QA（实习 + 项目 → 前沿技术）

> 依据《个人简历.pdf》中的 3 段经历整理：SmartX（QEMU 快照/按需恢复）、龙芯（RDMA 分布式共享内存）、veRL/AReaL 异步 post-training + 推测解码。 每部分分为：**A. 项目本身必问**（守住事实边界）→ **B. 底层原理深挖** → **C. 市场前沿扩展** → **D. 与 AI Infra 的桥接话术**。
>
> ⚠️ 事实边界提醒（来自腾讯 IEG 一面复盘）：简历里“对应后训练 checkpoint”“推理服务内存按需加载”是**关联思考**，不是实际做过的工作。被问到时先说“这是我把项目能力迁移到 AI 场景的思考”，再展开，避免被判夸大。

目录

1. SmartX：QEMU postcopy / 在线快照 / 按需恢复
2. 龙芯：RDMA 异步通信 + 分布式共享内存
3. veRL / AReaL：异步 post-training 与资源分配
4. 推测解码在 rollout 中的应用
5. 跨项目综合题 / 开放题
6. 速记表

一、SmartX：QEMU postcopy / 在线快照 / 按需恢复

A. 项目本身必问

Q1：一句话介绍你在 SmartX 做了什么？

A：在 QEMU 中基于 userfaultfd 写保护（UFFD-WP）实现后台在线快照：快照期间虚拟机继续运行，首次写某页前先把旧页内容拷出再放行，从而得到“快照开始时刻”的一致性内存镜像；另外验证了基于页索引 + userfaultfd 缺页线程 + blockdev/iSCSI 的大内存按需恢复链路。指标：16GB 内存、10%–90% 四档负载，平均停机 97ms，快照阶段 TPS 保持率 81.2%；4GB/8GB hugepage 场景按需恢复比普通文件恢复快约 20%–24%。

Q2：传统快照（savevm）和后台快照区别？

A：

	savevm（停机快照）	precopy 式 live snapshot	UFFD-WP 后台快照
VM 是否暂停	全程暂停	运行，最后短暂停	只在开始/结束短暂停
一致性时刻	暂停时刻	**结束**时刻	**开始**时刻
内存数据量	1×	可能多轮重传脏页，> 1×	严格 1×（每页只写一次）
风险	停机长	高脏页率下不收敛	写密集时 guest 写阻塞 → TPS 下降

UFFD-WP 本质是**用户态的写时复制（COW）**：不需要 fork，也不需要多轮脏页迭代，天然收敛。

Q3：UFFD-WP 后台快照完整流程？

A：

1. 暂停 vCPU，保存设备状态（CPU 寄存器、设备、元数据）；
2. 对 guest RAM 注册 userfaultfd（`UFFDIO_REGISTER_MODE_WP`），用 `UFFDIO_WRITEPROTECT` 把全部页设为写保护；
3. 恢复 vCPU；
4. 后台迁移线程顺序扫描页，写出到快照文件后解除该页写保护；
5. guest 写到一个尚未保存的页 → 触发 WP fault → fault 处理线程读到事件 → **优先**保存该页 → 解除写保护并唤醒 vCPU；
6. 全部页保存完成 → 结束。

Q4：停机 97ms 主要花在哪？如何继续降低？

A：主要在设备状态保存、全量写保护设置（大量页表项修改 + TLB 刷新）、vCPU 同步暂停/恢复。优化方向：

- 使用大页（2MB/1GB）减少页表项数量；
- 写保护设置并行化/分区间设置；
- 设备状态异步保存或延后（vhost 等设备的 dirty 处理）；
- 先写保护再停机（需要处理好一致性窗口）。

Q5：TPS 保持率 81.2% 意味着什么？损失来自哪里？

A：损失来自：①guest 写 fault 陷入 + 用户态处理 + 唤醒的延迟（每次 μs 级）；②后台写盘与业务争抢 IO 带

宽/CPU；③写保护导致的 TLB 失效。负载越写密集损失越大。优化：fault 页优先级队列、合并相邻页批量保存、快照写入限速/IO 隔离、预先保存热页（基于历史脏页统计）。

Q6：postcopy 迁移和 precopy 迁移区别？你为什么会涉及 postcopy?
A：

- precopy：先迭代传脏页，脏页够少后停机传剩余 → 写密集时**不收敛**。
- postcopy：先传设备状态让目的端立即运行，缺页时通过 userfaultfd（MISSING 模式）向源端请求页，后台同时推送剩余页 → **必收敛**，但目的端缺页有网络延迟，且源、目的端任一端挂掉则 VM 丢失（无完整副本）。
- 常见做法：precopy 若干轮后切 postcopy（hybrid）。
- 按需恢复与 postcopy 机理一致：只是页面来源从“源端网络”换成“快照文件/iSCSI 块设备”。

Q7：按需恢复为什么 hugepage 场景更快?
A：①一次缺页填充 2MB，缺页次数降 512×，陷入开销显著下降；②页索引更小、查找更快；③顺序读块设备更利于预读、IO 合并；④TLB 命中率高。代价是单次缺页延迟变大、写放大（填充整个大页）。

Q8：页索引是什么作用?
A：记录“guest 物理页号 → 快照文件偏移”的映射（可能还有零页/重复页标记），使缺页线程 O(1) 定位数据，避免顺序扫描快照流格式（QEMU 原生迁移流是顺序流，不支持随机读，因此需要额外索引或 `mapped-ram` 这类固定偏移格式）。

B. 底层原理深挖

Q9：userfaultfd 有哪几种模式?
A：

模式	触发	解决命令	用途
MISSING	访问未映射页	`UFFDIO_COPY` / `UFFDIO_ZEROPAGE`	postcopy、按需恢复、懒加载
WP（Linux 5.7+）	写写保护页	`UFFDIO_WRITEPROTECT` 清除	后台快照、用户态脏页跟踪、GC
MINOR（5.13+）	页在 page cache 但未映射	`UFFDIO_CONTINUE`	共享内存/hugetlbfs 的 postcopy（如 Google 的 VM 迁移）
异步 WP + `PAGEMAP_SCAN`（6.7+）	不发事件，内核直接解除并标脏	ioctl 扫描	低开销脏页跟踪（CRIU、游戏反作弊/模拟器）

Q10：userfaultfd 和 SIGSEGV + mprotect 方案相比优势?
A：mprotect 是 VMA 粒度，频繁拆分 VMA 开销大；信号处理在出错线程上下文执行、不可重入、难以多线程扩展；UFFD 是页粒度、事件通过 fd 异步投递给专门线程，支持原子“拷贝+映射”（UFFDIO_COPY 避免“先映射后填充”的竞态）。

Q11：KVM 下 guest 写一个页，UFFD-WP 是怎么生效的?
A：guest 写 → EPT/NPT 映射来自 QEMU 进程的 host 虚拟地址 → host 页表项被写保护 → KVM 在处理 EPT

violation 时调用 GUP（get_user_pages）写访问 → 触发 host 侧 handle_mm_fault → uffd WP 事件 → vCPU 线程阻塞直至用户态解除。所以本质依赖 host 页表，而 KVM 通过 MMU notifier 保持 EPT 与 host 页表一致。

Q12：KVM 脏页跟踪机制有哪些?
A：

- **dirty bitmap**：写保护 + 位图，`KVM_GET_DIRTY_LOG`，大内存下扫描位图开销大；
- **dirty ring**（Linux 5.11+）：每 vCPU 一个环，记录脏页 GFN，开销与脏页数成正比；
- **PML**（Intel Page Modification Logging）：硬件自动记录脏页 GPA 到缓冲区，减少写保护 fault；
- **manual protect / 分块清除**：`KVM_CLEAR_DIRTY_LOG` 降低一次性清除开销。

Q13：fork 也能做 COW 快照（如 Redis RDB），为什么 QEMU 不用?
A：fork 复制整个进程页表（大内存下 fork 本身就要几十到几百 ms）；vCPU 线程、vhost、VFIO 设备状态无法随 fork 正确复制；KVM fd 在子进程语义不对。UFFD-WP 只针对 guest RAM 区域，可控粒度更细。

Q14：写时复制（COW）在 Linux 中如何实现?
A：fork 时父子页表项都设为只读并增加 page 引用计数 → 写触发 page fault → `do_wp_page` 判断引用计数 >1 则分配新页拷贝，=1 则直接改为可写。大页 COW、KSM 合并页的 COW 同理。

Q15：大页有哪些种类？虚拟化里注意什么?
A：hugetlbfs（预留、不可换出，QEMU `-mem-path`）、THP（透明大页，可能有 khugepaged 合并开销与延迟抖动）、1GB 大页。虚拟化注意：EPT 也要大页才能减少二维页表遍历（最坏 24 次访存）；迁移/快照中大页的写保护和脏页粒度变粗导致写放大，postcopy 需要整页原子放置（UFFDIO_COPY 大页对齐）。

C. 市场前沿扩展

Q16：QEMU 迁移近几年有哪些新特性?
A：

- **multifd**：多通道并行传输内存页，配合压缩（zstd/zlib/QPL/QAT/UADK 硬件卸载）；
- **postcopy-preempt**：独立通道传缺页请求的紧急页，避免被后台大流量阻塞，显著降低 postcopy 缺页延迟；
- **background-snapshot** 能力（即你做的方向）；
- **mapped-ram**：页按固定偏移写入文件，支持并行写和随机读 → 天然适配按需恢复；
- **VFIO 设备迁移**（v2 协议）：支持 GPU/网卡等直通设备的状态保存和脏页跟踪（依赖厂商驱动，如 NVIDIA vGPU、mlx5 VF）；
- **CPR（CheckPoint and Restart）**：在 QEMU 升级时保留 guest 内存（memfd）原地重启，实现热升级。

Q17：GPU 状态怎么做快照/迁移？（AI 场景高频）
A：

- **NVIDIA cuda-checkpoint**（驱动 550+）：把进程的 CUDA 状态（显存、上下文）挂起并转移到主机内存，再配合 **CRIU** 做整个进程的 checkpoint/restore；
- **VFIO 迁移 / vGPU 热迁移**：hypervisor 层通过驱动导出设备状态和显存脏页；
- 应用层：PyTorch 的 `state_dict` 级 checkpoint 仍是主流，兼容性最好；
- 难点：显存数百 GB、无硬件写保护脏页跟踪、NCCL 通信器等外部状态无法透明迁移。

Q18：大模型训练 checkpoint 的前沿做法？（与你项目的桥接点）
A：

- **异步 checkpoint**：先 D2H 拷贝到 pinned 内存（短暂阻塞），后台线程再落盘 —— PyTorch DCP `async_save`、Megatron 分布式异步 ckpt；本质和后台快照一样是“捕获一致时刻 + 后台持久化”。
- **分布式/分片 checkpoint**：每 rank 存自己的分片，支持 reshard（改变并行度重新加载），如 PyTorch DCP、ByteDance 的 ByteCheckpoint。
- **内存/邻居备份 checkpoint**：把 ckpt 存在其他节点 CPU 内存（如 Gemini，SOSP’23），故障恢复走网络而不是存储，可做到每个 iteration 级别的 checkpoint。
- **增量/差分 checkpoint**：只存变化部分（对优化器状态效果有限，对 LoRA/embedding 有效）。
- **一致性**：必须在 optimizer.step 之间的边界取，与 UFFD-WP“开始时刻一致”同构；若 D2H 期间继续训练会修改参数，需要 COW 或双缓冲。

Q19：推理服务冷启动/模型快速加载有哪些技术？（按需恢复的桥接点）
A：

- **safetensors + mmap**：零拷贝、懒加载，只有访问到的 tensor 才触发缺页读入 —— 就是 OS 级“按需恢复”；
- **ServerlessLLM（OSDI’24）**：多级存储（SSD→DRAM→GPU）的加载优化 + 本地性感知调度；
- **GPUDirect Storage / 并行流水加载**：NVMe 直接 DMA 到显存，分块流水 H2D；
- **Run:ai Model Streamer、fastsafetensors** 等并发读取加载器；
- **vLLM sleep mode**：level 1 把权重 offload 到 CPU、丢弃 KV cache，level 2 连权重也丢弃，唤醒时快速恢复 —— RL 中 rollout/训练共卡切换的核心手段；
- **CUDA Graph / torch.compile 缓存**：减少启动编译时间。

Q20：KV cache 的分层存储与 offload 和你的工作有什么联系？
A：KV cache offload（LMCache、Mooncake Store、vLLM KV connector、SGLang HiCache）本质是“热数据在显存，冷数据在 CPU/SSD/远端，按需回取 + 预取”，与按需恢复的“页索引 + 缺页加载 + 预取”是同一类问题：索引结构（按 token block 哈希前缀）、粒度选择（page/block 大小对应大页权衡）、回取延迟隐藏（与计算 overlap）。

Q21：CXL 内存对虚拟化/AI 有什么影响？
A：CXL.mem 提供可扩展的远端内存（延迟约本地 DRAM 的 2 倍），用于内存扩展与池化；迁移时可以共享内存池而无需拷贝；AI 场景可用作 KV cache/参数的中间层。需要 OS 层的分层内存管理（NUMA 节点形式暴露，TPP、内核 memory tiering 自动冷热迁移）。

D. 桥接话术

> “我的工作本质是**大内存状态的一致性捕获与按需恢复**：快照阶段用写保护实现用户态 COW，在不长时间暂停业务的前提下拿到一致镜像；恢复阶段用缺页驱动 + 索引 + 大页把恢复做到按需。这套思路和训练的异步 checkpoint（一致时刻 + 后台持久化）、推理的 mmap 懒加载与 KV cache 分层（按需取 + 预取）是同构的。我在实际工作中没有直接做 AI 场景，但如果去做 checkpoint 或模型加载，我能迅速把 fault 粒度、预取、IO 隔离这些经验迁移过去。”

二、龙芯：RDMA 异步通信 + 分布式共享内存

A. 项目本身必问

Q22：M-JIAJIA 是什么？你做了什么？
A：JIAJIA 是中科院计算所的软件分布式共享内存（DSM）系统，基于 home-based、scope consistency，用页保护（mprotect + SIGSEGV）检测远程访问。我为其接入 UDP/RDMA 通信层：基于 RDMA CM 建链、内存注册、RC Send/Recv 实现消息通信；用消息队列解耦收发线程与处理线程，异步事件通知（completion channel）替代忙轮询；把通信流水线拆成可并行阶段。双机 RDMA 较千兆 UDP，Water-1728、LU-2048 总耗时降低 42.52%、64.72%，并加入远程预取减少取页与同步等待。

Q23：为什么 LU 的提升比 Water 大？
A：LU 分解访存跨页、远程取页和 barrier 更频繁，是通信受限型；Water（分子动力学）计算占比更高、通信相对少。RDMA 降低的是每次通信的延迟与 CPU 开销，因此通信越密集收益越大。（注意：千兆 UDP 对比本身带宽差距很大，面试时要主动说明对比基线，避免被质疑“对比不公平”；可补充说明同网卡 RoCE vs IPoIB/TCP 的对比思路。）

Q24：为什么用 Send/Recv 而不是 RDMA Write/Read？
A：DSM 协议是请求-响应消息语义（取页请求、diff、锁、barrier），Send/Recv 天然带通知、接收方可知道消息到达；单边 Write 需要额外的通知机制（Write with Immediate 或轮询标志位）。可进一步优化：页数据用 RDMA Read 直接从 home 节点拉取（不打扰远端 CPU），控制消息用 Send。

Q25：消息乱序、流控、同步怎么做？（腾讯面试被问过，答得不完整）
A：

- **顺序**：RC QP 内保证按序可靠交付；跨 QP（多连接/多线程）无序 → 消息带序号/会话 ID，接收端按需重排或保证协议对顺序不敏感（取页请求幂等）。
- **流控**：Recv 未预投递会导致 RNR（Receiver Not Ready）重试 → ①预投递足够 Recv WR 并及时补充；②基于 credit 的流控：接收端回传可用 buffer 数，发送端无 credit 则排队；③使用 SRQ 共享接收队列，按水位线补充。发送端也要限制未完成 WR 数（不超过 SQ 深度，并按 CQ 完成回收）。
- **同步**：每连接一个发送队列减少锁竞争；跨线程用无锁 SPSC/MPSC 环形队列，注意内存序（release/acquire）；CQ 由专门线程处理，通过 eventfd/条件变量通知处理线程。
- **背压**：处理线程慢时队列高水位 → 停止补充 credit，让发送端自然阻塞。

Q26：异步事件通知是怎么实现的？和轮询比较？
A：`ibv_create_comp_channel` + `ibv_req_notify_cq` → 在 channel fd 上 `poll/epoll` 或 `ibv_get_cq_event` 阻塞 → 被唤醒后 `ibv_ack_cq_events` 并重新 arm，再循环 `ibv_poll_cq` 把 CQ 排空（必须先 re-arm 再 poll，避免丢事件）。事件模式省 CPU 但有中断+唤醒延迟（几 μs）；忙轮询延迟最低但占满核。常用混合策略：先 poll 一段时间，空闲后转事件模式。

Q27：远程预取怎么做？有什么风险？
A：根据访问模式（顺序/步长）在缺页时顺带请求后续页，或在 barrier 后批量拉取本节点即将访问的页。风险：预取错误浪费带宽；预取的页在使用前被其他节点修改导致失效（需要按一致性协议正确处理 invalidation）。

B. 底层原理深挖

Q28：RDMA 为什么快?

A：内核旁路（用户态直接 doorbell 网卡）、零拷贝（网卡 DMA 读写注册内存）、协议卸载（传输层在网卡硬件实现）、单边操作不需要远端 CPU 参与。

Q29：RDMA 核心对象?

A：PD（保护域）、MR（内存注册，pin 住内存并建立 IOVA 映射，返回 lkey/rkey）、QP（SQ+RQ）、CQ、SRQ、AH（UD 用）。QP 状态机：RESET → INIT → RTR → RTS。RDMA CM 帮你完成地址解析、路由解析、交换 QP 号/GID 并驱动状态迁移。

Q30：QP 类型对比?

A：

类型	可靠性	支持操作	特点
RC	可靠有序	Send/Recv/Write/Read/Atomic	最常用，连接数 N^2 扩展性差
UC	不可靠	Send/Write	少用
UD	不可靠，MTU 限制	Send/Recv	一对多，扩展性好（eRPC、FaSST）
XRC / DCT	可靠	—	减少连接数（DCT 是 Mellanox 动态连接）

Q31：InfiniBand、RoCEv2、iWARP 区别?

A：IB 是专用网络（链路层基于 credit 无损）；RoCEv2 是 RDMA over UDP/IP 以太网，依赖 **PFC** 做无损、**ECN + DCQCN** 做拥塞控制；iWARP 基于 TCP，较少用。RoCE 的问题：PFC 风暴、死锁、头阻塞 → 业界推进 lossy RoCE、IRN、以及 UEC。

Q32：内存注册的开销如何优化?

A：注册需要 pin 页+写网卡 MTT，代价高 → 预注册大 buffer 池、注册缓存（registration cache）、使用 ODP（On-Demand Paging，网卡支持缺页）、大页减少 MTT 表项。

Q33：DSM 的一致性模型？JIAJIA 采用什么?

A：顺序一致性（代价高）→ 释放一致性（RC）→ 懒惰释放一致性（LRC，TreadMarks）→ 作用域一致性（ScC，JIAJIA）：只保证在同一把锁保护的作用域内可见，acquire 时使对应页失效，release 时把修改（diff）发回 home 节点。home-based 让每页有固定 home，取页只需访问 home，减少 diff 链。

Q34：多写者协议中的 twin/diff 是什么?

A：第一次写某页前复制一份 twin，release 时比较页与 twin 生成 diff，只发送修改部分，允许多个节点并发写同一页的不同部分（缓解伪共享）。

C. 市场前沿扩展

Q35：GPUDirect RDMA / GPUDirect Storage 是什么?

A：GDR 让网卡直接 DMA 读写 GPU 显存（通过 nvidia-peermem/dma-buf 注册显存为 MR），绕开 CPU 内存中转；关键是 PCIe 拓扑——网卡与 GPU 在同一 PCIe Switch 下性能最好，跨 CPU socket 性能明显下降。GDS 让 NVMe 直接 DMA 到显存。

Q36：NCCL 的通信算法和协议?
A：算法：Ring（带宽最优，延迟 O(N)）、Tree（双二叉树，延迟 O(logN)）、CollNet/SHARP（交换机内归约）、NVLS（NVSwitch 内置归约，NVLink SHARP）。协议：Simple（大消息高带宽）、LL（8B 数据+8B 标志，低延迟）、LL128。传输：节点内 P2P/NVLink/SHM，节点间 IB/RoCE（GDR）。

Q37：NVLink / NVSwitch 和 PCIe、RDMA 网络的关系?
A：节点内 scale-up：NVLink（H100 每卡 900GB/s 双向），NVSwitch 全互联；GB200 NVL72 把 72 卡做成一个 NVLink 域。节点间 scale-out：IB/RoCE 400G/800G 网卡，每卡一张。层次化并行：TP/EP 放在 NVLink 域内，PP/DP 跨节点。

Q38：昇腾上对应的是什么？（HCCL vs NVLink）
A：HCCL 是华为集合通信库（对标 NCCL），HCCS 是节点内互联（对标 NVLink），节点间 RoCE。注意概念层次：NCCL/HCCL 是**库**，NVLink/HCCS 是**互联硬件**。CloudMatrix 384 超节点用 UB（统一总线）把 384 颗 NPU 组成大 scale-up 域。

Q39：MoE 的 all-to-all 通信前沿：DeepEP / IBGDA / NVSHMEM?
A：

- MoE 的 EP 需要 dispatch/combine 两次 all-to-all，token 不均衡、小消息多；
- **DeepEP**（DeepSeek 开源）：高吞吐 kernel（训练/prefill，NVLink + RDMA 两级转发）和低延迟 kernel（decode，纯 RDMA），支持通信-计算 overlap 且不占 SM（hook 方式）；
- **IBGDA**（GPU 发起 IB 通信）：GPU 线程直接写网卡 doorbell，不经过 CPU proxy，降低小消息延迟；
- **NVSHMEM**：GPU 端的 PGAS 编程模型，kernel 里直接 put/get 远端显存 —— 与你做的 DSM “全局地址空间”思想同源。

Q40：PD 分离推理中的 KV cache 传输?
A：Prefill 节点算好 KV cache 后通过 RDMA 传给 Decode 节点。代表：**Mooncake**（Kimi，Transfer Engine 支持多网卡聚合、拓扑感知，KVCache 为中心的调度）、**NIXL**（NVIDIA Dynamo 的传输库，统一 GPU/CPU/存储后端）、vLLM/SGLang 的 KV connector。核心问题：按层流水传输与计算 overlap、多网卡带宽聚合、KV 布局转换（不同 TP 度）、注册内存管理。

Q41：RL 训练中的权重同步通信?
A：训练端更新完权重要同步给推理引擎：共卡时走 CUDA IPC 原地共享；分离部署时走 NCCL broadcast 或 RDMA（例如通过 checkpoint-engine 类工具、Mooncake Transfer Engine 做 P2P 传输），千亿模型要做到秒级 —— 需要分桶、流水、多网卡聚合，以及推理端在线 reload 权重。

Q42：Ultra Ethernet（UEC）是什么?
A：UEC 联盟发布的面向 AI/HPC 的以太网传输规范：多路径 packet spraying、乱序交付（应用层/传输层容忍乱序）、新拥塞控制、可选无需 PFC，旨在替代 RoCEv2 的无损网络依赖。与之相关的还有 AWS 的 SRD（EFA）、Google Falcon。

Q43：AI 集群网络拥塞与负载均衡问题?
A：AI 流量是少量大象流、同步突发，ECMP 哈希冲突严重 → 方案：自适应路由、packet spraying、按 QP 多路径、rail-optimized 拓扑（同号 GPU 接同一 leaf）、集合通信拓扑感知调度。

D. 桥接话术

> “M-JIAJIA 让我把 RDMA 的完整链路走了一遍：CM 建链、MR 注册、QP 状态机、CQ 事件、流控与背压。DSM 的’全局地址空间 + 按需取页 + 预取’在今天的 NVSHMEM、KV cache 远端池化里都还在用；而 rollout 与训练端的权重同步、PD 分离的 KV 传输，本质都是在 RDMA 上做大块数据的低延迟、可流水的传输，我对其中的注册内存管理、流控和通信线程解耦有一手经验。”

三、veRL / AReaL：异步 post-training 与资源分配

A. 项目本身必问

Q44：介绍项目背景与结论?

A：在昇腾 NPU 八卡环境上分析 GRPO 训练流水线（rollout 生成 → reward → 参考模型/旧策略 logprob → actor 更新 → 权重同步），调研同步与异步 post-training 相关论文并在八卡流水线上对比实验；通过异步获得加速，同时把推理/训练卡数从 2/6 调整为 4/4，平均步耗时缩短 25.51%。

Q45：你是怎么发现 2/6 不合理的?

A：profiling 发现 rollout 是瓶颈：生成阶段（尤其长尾样本）耗时远大于训练阶段，2 卡推理时训练卡长时间空闲等待数据。异步模式下理想状态是**生产者（rollout）与消费者（训练）速率匹配**：吞吐 ∝ min(rollout 吞吐, 训练吞吐)，所以把卡从训练端挪给推理端，使两端时间接近。可用的思路：分别测 rollout 每卡吞吐 Tg 与训练每卡吞吐 Tt，取 Ng · Tg ≈ Nt · Tt。

Q46：同步 RL 和异步 RL 有什么本质区别?

A：

- 同步（on-policy）：rollout 用的权重就是当前要更新的权重，生成 → 训练严格串行，GPU 交替空闲；
- 一步异步（one-step off-policy）：训练第 k 步时 rollout 已用第 k−1 版权重生成下一批数据，生成与训练重叠；
- 完全异步（AReaL）：rollout worker 持续生成，训练端攒够一批就训练，权重更新后推理端随时（可中断进行中的生成）加载新权重；
- 代价：数据**陈旧度（staleness）** → off-policy 偏差，需要算法修正。

Q47：AReaL 如何控制陈旧度和 off-policy 偏差?

A：①限制最大陈旧度 η（数据生成所用版本与当前版本差 ⩽ η，超了暂停生成）；②**decoupled PPO 目标**：把行为策略（生成数据的旧版本）与近端策略（proximal policy）解耦，用重要性权重修正；③interruptible generation：新权重到达时中断正在生成的序列，丢弃 KV cache 用新权重继续生成（一条序列可能由多个版本生成）。

Q48：GRPO 和 PPO 的区别?

A：PPO 需要 critic（价值模型）估计 advantage（GAE）；GRPO 去掉 critic，对同一 prompt 采样 G 个回答，用组内 reward 标准化作为 advantage：$\hat{A}_i = (r_i - \text{mean}) / \text{std}$，配合 clipped ratio 与 KL 惩罚。省一个模型的显存与计算，但需要每个 prompt 多次采样 → rollout 开销更大，长尾更明显。

Q49：GRPO 之后有哪些改进?

A：

- **DAPO**：Clip-Higher（上下界解耦，防熵塌缩）、动态采样（过滤全对/全错的组）、token 级 loss 归一化、超长样本的软惩罚；
- **Dr.GRPO**：去掉长度归一化和 std 归一化带来的偏差；
- **GSPO**（Qwen）：序列级重要性比率与裁剪，MoE 训练更稳定；
- **RLOO / REINFORCE++**：其他无 critic 的 baseline 方式；

- 针对训推不一致：**TIS（截断重要性采样）**修正推理引擎与训练引擎 logprob 不一致（不同 kernel/精度导致）。

B. 底层原理深挖

Q50：veRL 的架构（HybridFlow）？

A：单控制器（single-controller，Ray driver 编排数据流）+ 多控制器（每个 worker 内 SPMD 执行 FSDP/Megatron/vLLM/SGLang）。**3D-HybridEngine**：actor 训练与 rollout 共用同一组卡（colocate），切换时在训练并行布局与推理并行布局之间 reshard 权重，推理时释放训练的显存（优化器 offload），训练时推理引擎 sleep。

Q51：colocate（共卡）与 disaggregate（分离）各自优缺点?

A：

	共卡（veRL 默认）	分离（AReaL / 异步）
资源利用	时分复用，无空闲但串行	并行，需要配比好否则一端空闲
切换开销	权重 reshard、显存 offload/onload	权重跨卡/跨机传输
长尾	整个集群等最长样本	rollout 长尾可被训练掩盖
适合	同步 on-policy、中小规模	异步、大规模、异构硬件

Q52：rollout 为什么是瓶颈？长尾怎么治理?

A：推理 decode 是访存受限，且 RL 推理（尤其 reasoning）输出长度分布极度长尾——少数样本几万 token，batch 内大部分序列结束后 GPU 利用率骤降。治理：

- **partial rollout**（Kimi K1.5）：超长样本截断，下轮继续生成；
- 异步/流式：先完成的样本先送去训练（StreamRL、AReaL）；
- 超量采样后丢弃最慢的部分（over-sampling）；
- 动态 batch / 连续批处理 + 负载均衡、按长度预测调度；
- **推测解码**：在小 batch 长尾阶段 GPU 算力空闲，正好适合推测解码（见第四部分）；
- 最大长度限制 + 超长惩罚。

Q53：权重同步怎么做?

A：共卡：训练布局 → 推理布局 reshard，CUDA IPC 共享句柄；分离：NCCL group broadcast（训练 rank0 或各 TP 分片并行发送），分桶流水；推理引擎通过 `update_weights` 接口加载。注意：MoE、量化推理（FP8）需要在同步时做格式转换。

Q54：昇腾 NPU 生态与迁移要点?

A：

- 软件栈：CANN（对标 CUDA）、AscendCL、torch_npu（PyTorch 适配）、MindSpeed（对标 Megatron）、vllm-ascend（vLLM 硬件插件）、HCCL；
- 算子：Ascend C 编程（对标 CUDA C），Cube 单元做矩阵、Vector 单元做向量；
- 迁移：算子映射与缺失算子补齐、图模式（torchair/静态图）提升性能、精度对齐（bf16/fp16 差异、累加顺序）、通信原语替换、profiling 工具（msprof / torch_npu profiler + MindStudio Insight）；
- 常见问题：动态 shape 导致重复编译、host 下发瓶颈、算子 fallback 到 CPU。

Q55：profiling 时你重点看什么？

A：各阶段时间线（generate / ref logprob / old logprob / update / sync）、卡的空闲比例、通信占比、下发（host bound）间隙、显存峰值。核心指标：每步耗时、tokens/s、MFU、rollout 与训练的时间比。

C. 市场前沿扩展

Q56：主流 RL 训练框架对比？

A：

框架	来源	特点
veRL	字节 Seed	HybridFlow，共卡，生态最广，支持 FSDP/Megatron + vLLM/SGLang，已加入异步/分离模式
AReaL	蚂蚁 + 清华	完全异步，可中断生成，陈旧度控制
OpenRLHF	开源社区	Ray + vLLM + DeepSpeed，简洁
slime	智谱/THUDM	Megatron + SGLang，GLM-4.5 系列训练使用
ROLL	阿里	大规模，多角色调度
NeMo-RL	NVIDIA	Megatron 生态
TRL	HuggingFace	易用，规模较小

Q57：Agentic RL（多轮/工具调用）带来什么新挑战？

A：rollout 包含环境交互（代码执行、搜索、浏览器），时间不确定，GPU 等待环境 → 需要异步 rollout、环境服务化与沙箱并发；多轮对话的 KV cache 复用（前缀缓存）；轨迹级 reward 与 token mask（工具返回内容不计算 loss）；长上下文。

Q58：训推一致性问题（training-inference mismatch）？

A：推理引擎（vLLM/SGLang，可能 FP8、不同 kernel、批量不确定性）算出的 logprob 与训练引擎不一致，导致“假 off-policy”，严重时训练崩溃。方案：重要性采样修正（TIS/MIS）、确定性推理（batch-invariant kernel）、训练与推理用同精度、用训练端重算 logprob。

Q59：大规模 RL 的资源调度趋势？

A：从固定比例到弹性：根据 rollout/训练实时吞吐动态调整卡数；异构（推理用性价比高的卡、训练用高端卡；或 NPU/GPU 混合）；rollout 用 PD 分离和 EP 大规模推理；“Rollout-as-a-Service”。

D. 桥接话术

> “这个项目我最核心的收获是**用 profiling 数据驱动资源配比**：先把 GRPO 每一阶段的时间拆出来，发现推理端是瓶颈、训练卡在等数据，于是从 2/6 调到 4/4 让两端速率匹配，步耗时降 25.51%。再往前一步就是异步：用陈旧度可控的 off-policy 换取生成与训练重叠。我也清楚异步的代价，需要 decoupled PPO/重要性采样去修正。”

四、推测解码在 rollout 中的应用

A. 项目本身必问

Q60：为什么在 RL rollout 里用推测解码?

A：rollout 尾部阶段 batch 很小、decode 访存受限，算力大量空闲；推测解码用空闲算力一次验证多个 token，把多步串行 decode 合成一步，正好缓解长尾。而且 RL 每步都要生成大量 token，加速 rollout 等于加速整个 post-training。

Q61：为什么 draft model 需要实时更新?

A：RL 训练过程中 target（actor）持续变化，固定的 draft 与 target 分布逐渐偏离，**接受率下降**，加速比随训练衰减甚至变成负收益。所以要在训练过程中用 actor 最新生成的数据（rollout 本身就是现成的蒸馏数据）在线训练 draft，让其跟随 target 漂移。

Q62：draft 在线更新怎么做？开销如何控制?

A：用 rollout 产生的序列（以及 target 的 hidden states/logits）作为训练数据，在训练间隙或空闲卡上训练 draft（如 EAGLE 头只是一层 decoder，训练开销远小于 actor）；更新频率与 actor 权重同步一起或每 N 步一次；损失函数：对 target 分布的 KL / 交叉熵（EAGLE 还加 feature 回归损失）。

Q63：推测解码会不会改变输出分布，影响 RL 的正确性?

A：标准的推测采样（rejection sampling）**无损**：以概率 $\min(1, p(x)/q(x))$ 接受 draft token，拒绝时从归一化的 $\max(0, p-q)$ 重采样，最终分布严格等于 target p。所以 on-policy 性质不变，只要 logprob 由 target 计算。（注意：如果用的是贪心/松弛接受规则，就不是无损的，需要说明。）

B. 底层原理深挖

Q64：推测解码加速比如何估计?

A：设每 token 接受率 α，草稿长度 γ，则每轮期望生成 token 数 $E = (1 - \alpha^{\gamma+1}) / (1 - \alpha)$。加速比 $\approx E / (1 + \gamma \cdot c)$，c 为 draft 单步与 target 单步耗时比。α 越高、c 越小越好；γ 存在最优值。

Q65：为什么大 batch 时推测解码收益下降?

A：大 batch 时 decode 已接近计算受限，验证 γ 个额外 token 的计算不再“免费”，且被拒绝 token 浪费算力。所以 RL 中应**动态开关/调整 γ**：batch 大时关闭或减小 γ，长尾小 batch 时开启并加大 γ。

Q66：推测解码有哪些主流方法?

A：

类别	方法	说明
独立 draft 模型	小模型（如 Qwen-0.5B 给 7B 做 draft）	简单，需同词表，额外显存
自身附加头	Medusa	多个并行解码头 + tree attention
特征级自回归	**EAGLE / EAGLE-2 / EAGLE-3**	用 target 的 hidden state 预测，EAGLE-2 动态树，EAGLE-3 多层特征融合 + 训练时测试（training-time test），接受率最高之一
原生 MTP	DeepSeek-V3 MTP、Qwen3-Next、GLM-4.5 等	预训练即带 MTP 模块，推理时直接当 draft
无模型	n-gram / prompt lookup / Suffix Decoding / Lookahead	利用重复片段，零训练，适合代码/RL 中重复模式
自推测	LayerSkip、跳层	用 target 的浅层当 draft

Q67：树形验证（tree attention）是什么?

A：draft 每一步保留多个候选，构成 token 树，target 一次前向用特殊 attention mask 同时验证整棵树，选最长被接受的路径。提升每轮接受长度，代价是验证 token 数增加。EAGLE-2 根据 draft 置信度动态扩展树。

Q68：推测解码与 KV cache、连续批处理的工程问题?

A：被拒绝 token 的 KV 需要回滚（paged attention 下只需回退 slot 指针）；batch 内不同序列接受长度不同，需要 ragged 处理；CUDA Graph 需要固定形状（按 γ 捕获多份）；draft 和 target 各自的 KV cache 显存开销。

C. 市场前沿扩展

Q69：RL 场景中推测解码的相关前沿工作?

A：2025 年起出现一批面向 RL rollout 的工作，核心都是“**长尾 + 自适应 draft**”：

- 利用 RL 训练中空闲资源在线训练 draft（如 MIT Han Lab 的 TLT 等工作，提出自适应 drafter 与按需开启推测）；
- 并发感知的推测策略（根据当前 batch 大小动态调整 γ 和是否开启）；
- 利用历史 rollout（同一 prompt 多 epoch 的相似回答）做 n-gram/后缀树检索式草稿，零训练开销；
- MTP 模块随 actor 一起训练，天然保持对齐。 （面试时说“调研过这类工作”即可，具体论文名记不准不要硬报。）

Q70：在线推测解码（Online Speculative Decoding）的一般思路?

A：在推理服务中收集用户查询下 draft 被拒绝的位置，用 target 输出持续蒸馏 draft，使其适应真实流量分布；RL 场景是它的特化——分布漂移来自 target 自身更新而不是流量变化。

Q71：推测解码与量化、MoE 的结合?

A：MoE 验证 γ 个 token 会激活更多专家，访存增加，加速比低于 dense；量化 draft（如 INT4 draft）降低 c；FP8 target 推理时要注意 draft 与 target 精度不同导致的接受率变化。

Q72：除了推测解码，还有哪些 decode 加速技术?

A：量化（W8A8/FP8/W4A16、KV cache 量化）、MLA 降低 KV cache、稀疏注意力（DeepSeek DSA、NSA）、

PD 分离、前缀缓存（RadixAttention）、CUDA Graph、算子融合（FlashAttention/FlashInfer/FlashMLA）、大 EP 降低单卡负载。

D. 桥接话术

> “推测解码这部分我抓的是 RL 特有的矛盾：rollout 长尾阶段 GPU 算力空闲，正好用来做推测验证；但 actor 一直在变，固定 draft 接受率会衰减，所以要用 rollout 数据在线更新 draft。同时推测采样是无损的，不破坏 on-policy。工程上还需要根据 batch 大小动态开关，大 batch 时推测是负收益。”

五、跨项目综合题 / 开放题

Q73：三段经历有什么共同主线？

A：**数据移动与资源利用率**：SmartX 解决大内存状态的一致性捕获与按需搬运；龙芯解决跨节点数据的低延迟搬运；RL 项目解决生成与训练之间的数据流与资源配比。共同方法论：先 profiling 拆时间 → 找瓶颈（停机、通信、空闲）→ 用异步/流水/按需把关键路径缩短。

Q74：千卡训练稳定性如何保障？（腾讯面试薄弱点）

A：

- **训前**：烧卡压测（GEMM/NCCL all-reduce 带宽测试）、健康检查（DCGM 诊断、ECC、NVLink、网卡、PCIe 链路）、慢节点筛查；
- **训中**：监控 XID 错误、ECC、温度/功耗、NVLink/IB 错误计数、loss/梯度范数异常（NaN、spike）、step 耗时（慢节点检测）；NCCL 超时/hang 检测（flight recorder 抓集合通信状态）；
- **故障处理**：自动隔离坏卡/坏节点、从备用池替换、从最近 checkpoint 自动恢复（in-memory checkpoint 缩短恢复时间）、弹性训练（调整 DP 度）；
- **事后**：堆栈采集、根因归类、更新黑名单。
- 关联：我的快照工作就是“快速一致性保存 + 快速恢复”，正是缩短故障恢复时间（MTTR）的手段。

Q75：如果让你设计 RL 系统中训练端到推理端的秒级权重同步，你会怎么做？

A：①训练端各 rank 按推理端 TP 布局预先切分好待发送分片（避免汇聚到 rank0）；②分桶（bucket）+ 多流流水：D2D 拷贝打包 → RDMA/NCCL 发送 → 推理端加载，三阶段 overlap；③多网卡并行，拓扑感知选择最近网卡（GDR）；④注册内存池复用，避免每次注册；⑤推理端双缓冲或分层切换，减少暂停服务时间；⑥量化格式在发送端完成转换。

Q76：异步训练中如何保证 checkpoint 一致性（rollout 状态 + 训练状态）？

A：训练状态（权重、优化器、数据迭代器、随机数种子）在 step 边界保存；rollout 端未完成的样本要么丢弃要么连同其版本号保存在 replay buffer 中；记录权重版本号使恢复后陈旧度可计算。思路与 UFFD-WP 快照相同：选一个“逻辑一致时刻”，其余后台异步持久化。

Q77：你对 AI Infra 方向的理解与规划？

A：AI Infra 的核心是把算力、显存、带宽这三类稀缺资源用满：训练侧是并行策略、通信 overlap、稳定性；推理侧是 KV cache 管理、调度、PD 分离、推测解码与量化；RL 侧是两者的结合与资源编排。我的优势在系统底层（虚拟内存、RDMA、虚拟化）和 RL 流水线分析，希望在推理框架/RL 系统方向深入，并补齐 CUDA kernel 和大规模集群经验。

六、速记表

关键词	一句话
UFFD-WP	用户态写保护 fault，实现后台快照的用户态 COW，Linux 5.7+
UFFD MISSING	缺页交给用户态填充，postcopy / 按需恢复 / 懒加载
UFFD MINOR	page cache 已有、只缺映射，用 CONTINUE 解决，适合共享内存
precopy vs postcopy	前者可能不收敛，后者必收敛但缺页延迟+双端失败风险
postcopy-preempt	缺页紧急页走独立通道，降低缺页延迟
dirty ring / PML	比位图更低开销的 KVM 脏页跟踪
cuda-checkpoint + CRIU	GPU 进程级 checkpoint/restore
DCP async_save	PyTorch 异步分布式 checkpoint
Gemini	内存级 checkpoint，邻居节点备份
vLLM sleep mode	释放/offload 权重与 KV，RL 共卡切换
RC/UD	可靠连接 vs 不可靠数据报
RNR	接收端未投递 Recv 导致的重试
PFC/ECN/DCQCN	RoCE 无损与拥塞控制
GDR	网卡直读写显存
NCCL Ring/Tree/NVLS	带宽最优 / 延迟最优 / 交换机内归约
DeepEP / IBGDA	MoE all-to-all 库 / GPU 直接发起 RDMA
Mooncake / NIXL	KV cache 为中心的分离式推理 / NVIDIA 推理传输库
UEC	面向 AI 的以太网传输，多路径+乱序
HCCL / HCCS / CANN	昇腾的集合通信库 / 互联 / 软件栈
GRPO	组内 reward 标准化作 advantage，无 critic
DAPO	Clip-Higher + 动态采样 + token 级 loss
GSPO	序列级重要性比率，MoE 更稳
HybridFlow	veRL：单控制器编排 + 多控制器执行 + 共卡 reshard
AReaL	完全异步 + 可中断生成 + 陈旧度控制 + decoupled PPO
partial rollout	超长样本截断续写（Kimi K1.5）
TIS	截断重要性采样，修正训推 logprob 不一致
推测采样	接受率 min(1,p/q)，拒绝从 max(0,p−q) 重采样，无损
E = (1−α^{γ+1})/(1−α)	每轮期望生成 token 数
EAGLE-3	多层特征融合 + training-time test 的 draft 头
MTP	多 token 预测模块，可直接做 draft（DeepSeek-V3）